



Conversational Interaction in WebGIS: A Prototype

Md Ashifur Rahaman¹ and Sujit Kumar Sikder²

¹Stuttgart Technical University of Applied Sciences, Germany

²Leibniz Institute of Ecological Urban and Regional Development, Germany



WebGIS

Users should define the input

correct layer, correct tool, required parameters, sequence of operations

WebGIS usability (Unrau, R. & Kray, C., 2021)

Tool-centred interactions often challenge situations where users have less flexibility to define the outcome.

LLM-WebGIS

User interaction **deterministic direct outcomes**

Show me parks nearby Dresden Hygienemuseum.

Geospatial NLI and LLM context (Naveen et al., 2024, Wang et al. 2024)

The challenge is not only understanding the sentence, but translating it into a safe, executable, and inspectable WebGIS action.

Ongoing discussions

Cloud dependency

Many systems rely on external LLM services, raising concerns about prompt privacy, governance, and vendor dependency.

Uncontrolled execution

Free-form model outputs are difficult to connect safely to map operations and geospatial services.

Limited transparency

Users often see the final result but not the decision source, validated action, or failure path.

Proposed Conceptual Framework

LLM supports interpretation;
validation and GIS execution remain
separate system responsibilities.





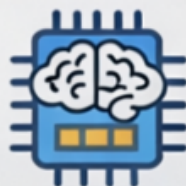
User Interaction Layer

Browser based WebGIS UI: map + chat interface



Natural Language Interpretation Layer

Local LLM: intent interpretation + action planning



Validation and Control Layer

Schema validation + safety checks + control logic



Geospatial Execution Layer

Deterministic spatial operations: map navigation,
WFS layer loading, distance/counts



Data and Service Layer

OGC Web Feature Services: example parking data
(MobiData Baden-Württemberg)



System architecture

Frontend WebGIS

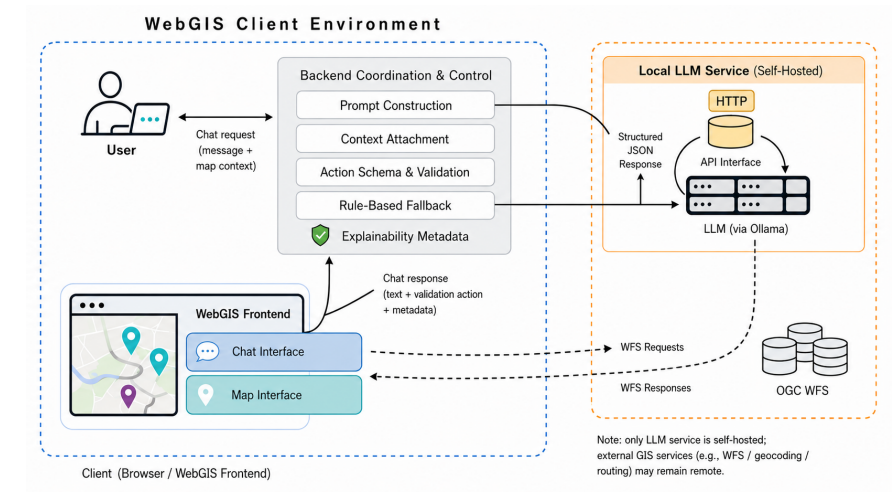
Map, chat interface, layers, and action execution

Backend coordination and control

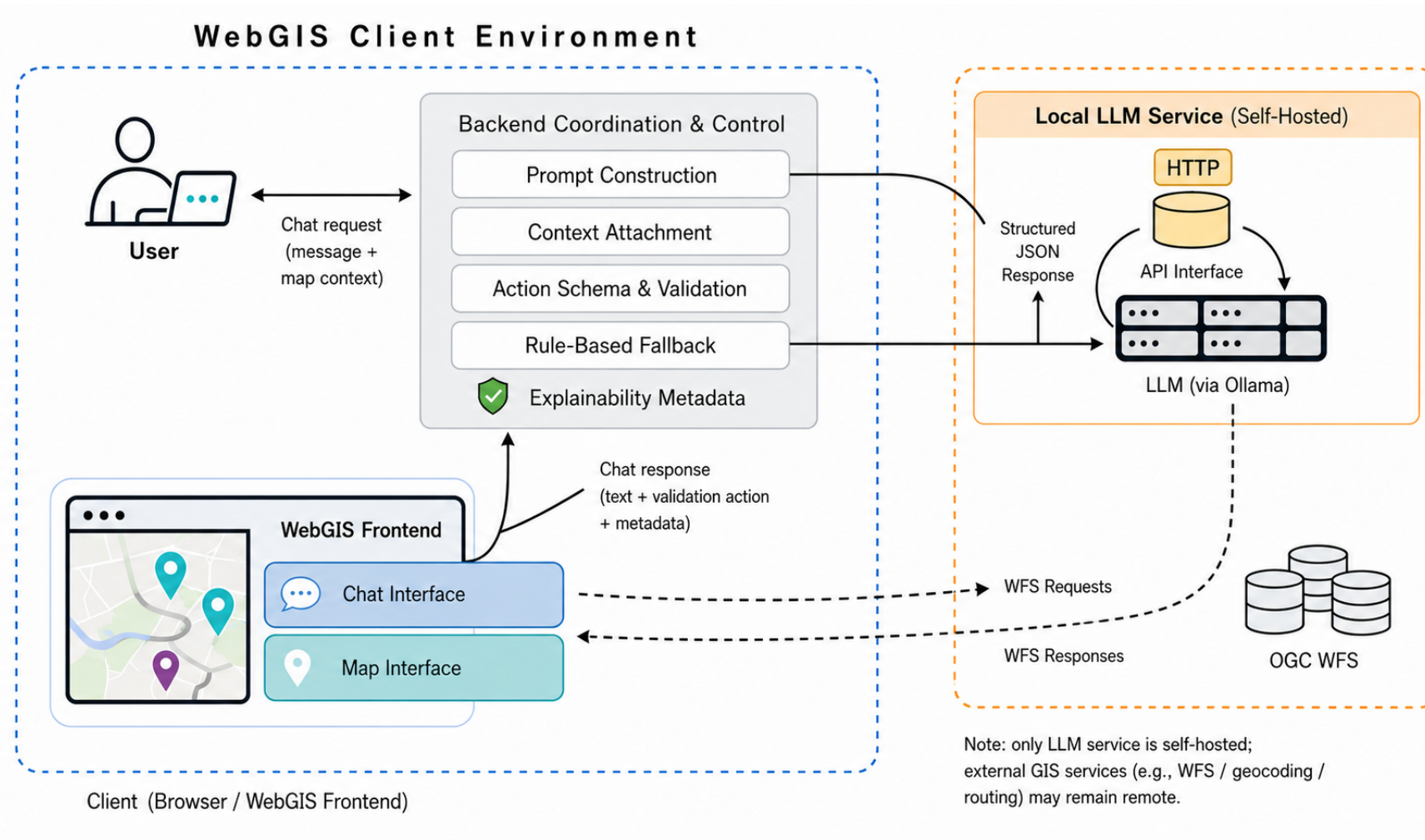
Context attachment, prompt construction, schema validation, fallback, and metadata.

Local LLM service via Ollama**

Converts the user request into a structured action proposal.



System architecture



Proto-type and Case study

Frontend: Leaflet, HTML, CSS, and JavaScript

Backend: Python and Flask

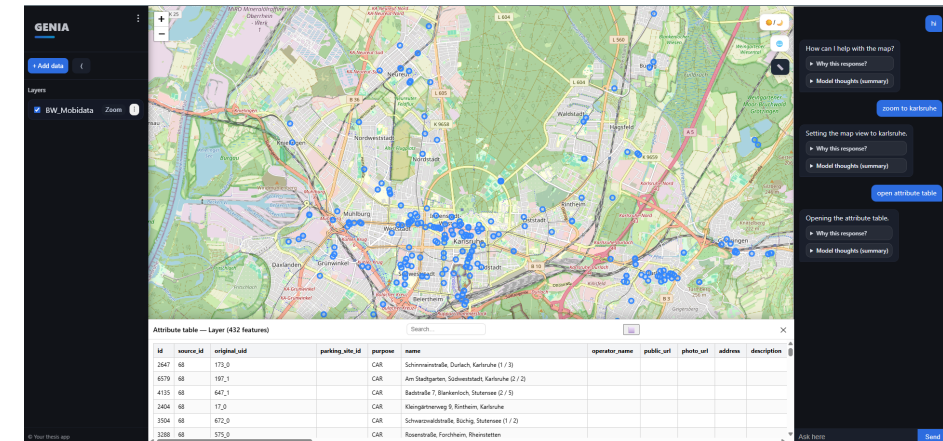
Local model service: Ollama, Qwen2.5 7B
Instruct (Q4_K_M)

Data access: OGC Web Feature Service

Actions: `setView` `loadWFS` `filterWithin`

`findByAttribute` `describeLayer` `countLayer`

`measureDistance` `openAttributeTable`



POI - MobiData, Baden-Württemberg, DE

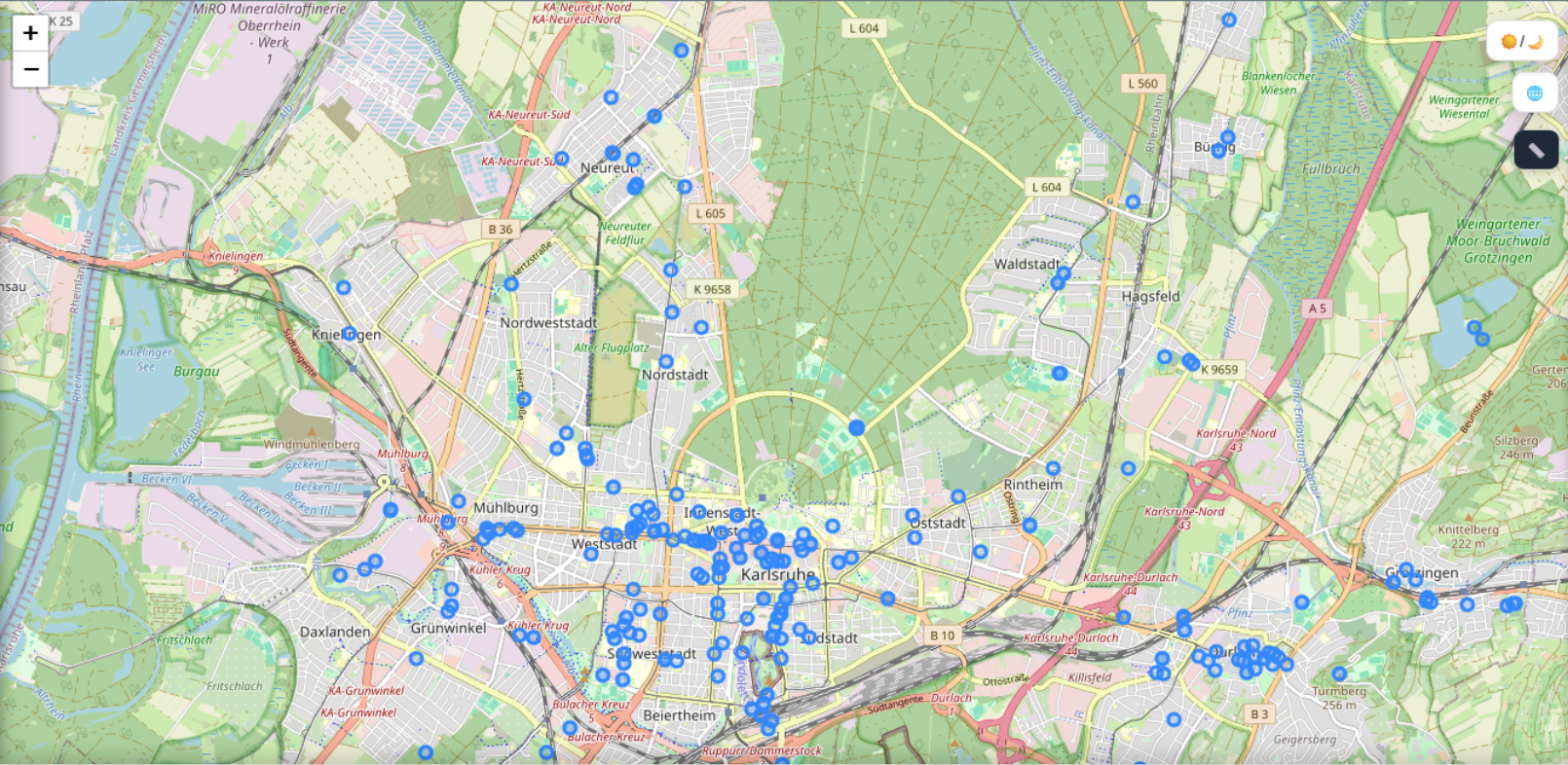
GENIA

+ Add data

Layers

☒ BW_Mobidata

Zoom



Attribute table — Layer (432 features)

id	source_id	original_uid	parking_site_id	purpose	name	operator_name	public_url	photo_url	address	description
2647	68	173_0		CAR	Schinnrainstraße, Durlach, Karlsruhe (1 / 3)					
6579	68	197_1		CAR	Am Stadtgarten, Südweststadt, Karlsruhe (2 / 2)					
4135	68	647_1		CAR	Badstraße 7, Blankenloch, Stutensee (2 / 5)					
2404	68	17_0		CAR	Kleingärtnerweg 9, Rintheim, Karlsruhe					
3504	68	672_0		CAR	Schwarzwaldstraße, Büchig, Stutensee (1 / 2)					
3288	68	575_0		CAR	Rosenstraße, Forchheim, Rheinstetten					

hi

How can I help with the map?

Why this response?

Model thoughts (summary)

zoom to karlsruhe

Setting the map view to karlsruhe.

Why this response?

Model thoughts (summary)

open attribute table

Opening the attribute table.

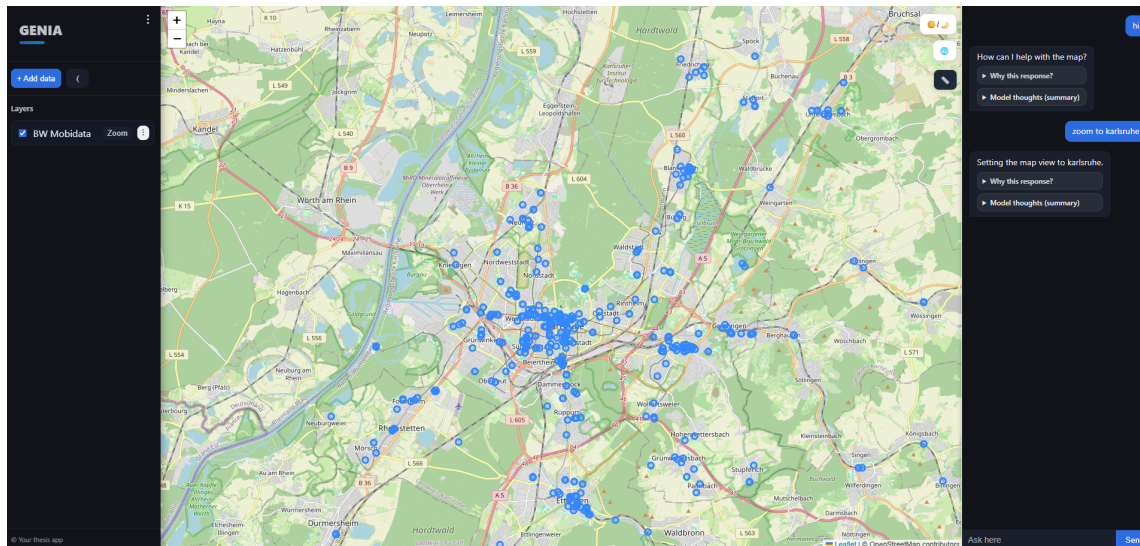
Why this response?

Model thoughts (summary)

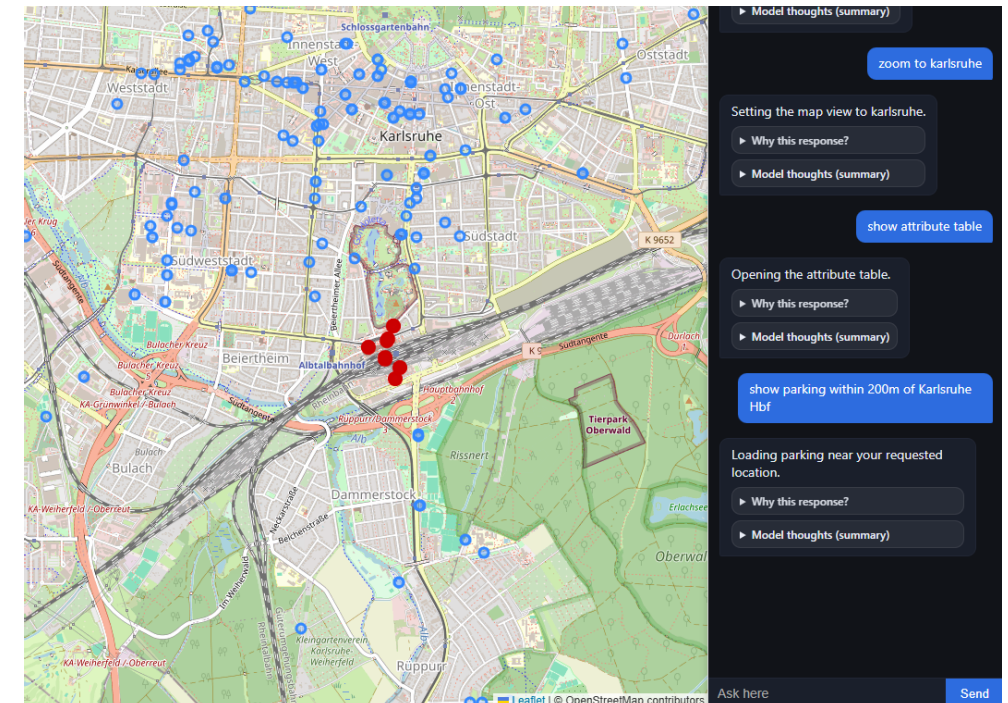
Ask here

Send

Example Task 1 — Load and filter



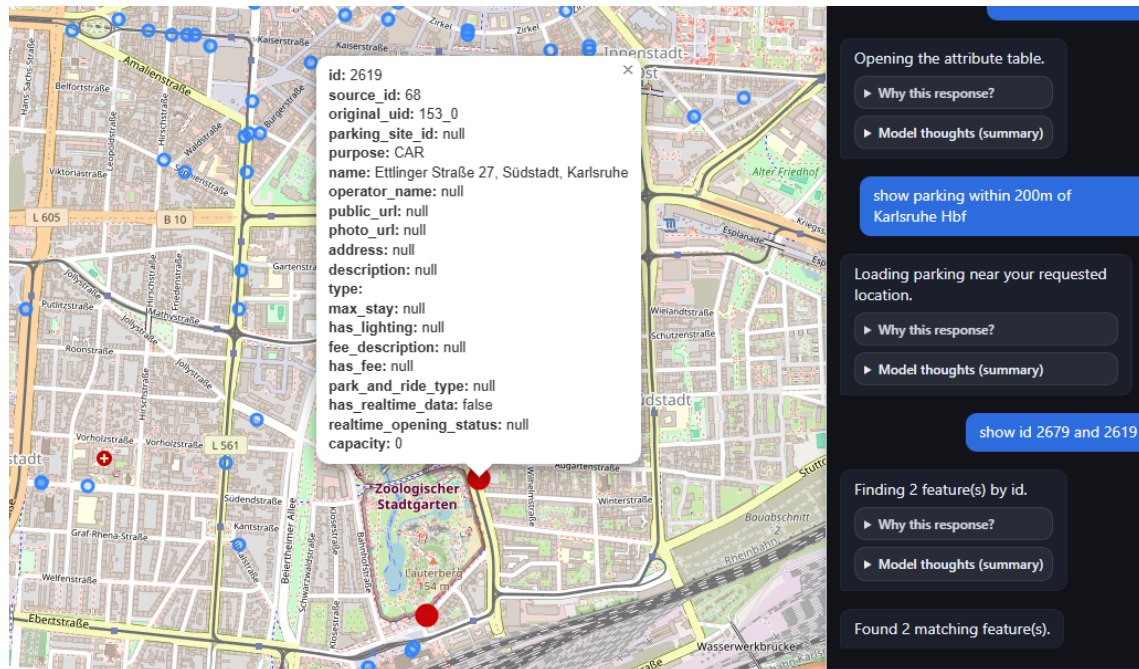
Parking facilities loaded from the MobiData BW WFS.



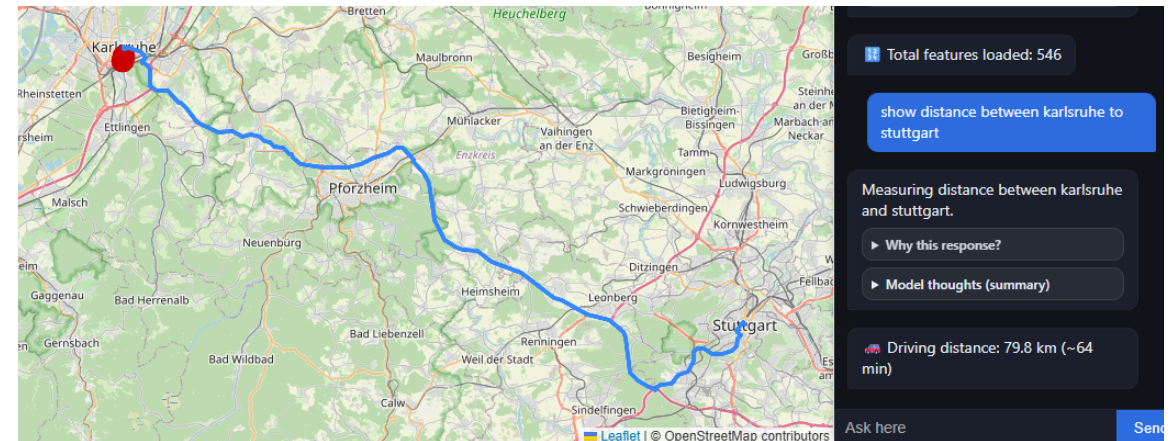
User Query: “Show parking within 200 m of Karlsruhe Hbf.”

The place name and radius are converted into a `filterWithin` action.

Example Task 2 — Retrieve, count and measure



Attribute-based feature retrieval and map/table linking.



Route and driving distance between Karlsruhe and Stuttgart.

The distance request is converted into a `measureDistance` action and visualised as a route.

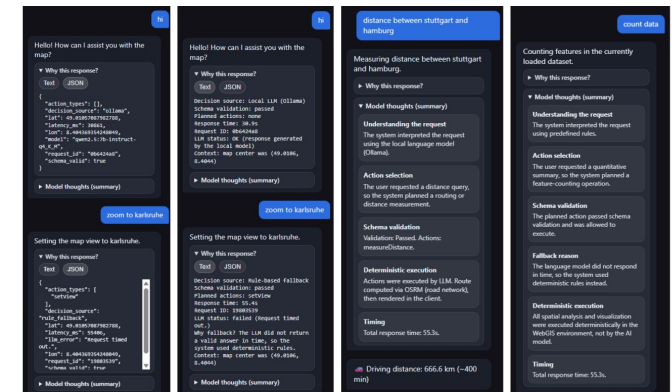
Explainability and failure handling

“Why this response?” metadata

The interface exposes:

- decision source: **local LLM or fallback**,
- proposed/executed action types,
- schema-validation status,
- response time,
- and error or timeout information.

Explainability is provided through system traces—not by exposing private model reasoning.



Evaluation results

Prototype was tested with **20 natural-language queries**.

Metric	Result
Expected task-level outcomes	16 / 20
Observed success rate	80%
Local LLM responses	11 / 20
Rule-based fallback responses	9 / 20
Latency range	15.9–56.7 s
Average latency	47.7 s

The system successfully operated within the bounded action scope; CPU-only local inference has hardware dependency.

Conclusion

This experimental demonstration illustrates a controlled integration of local LLMs into WebGIS.

- five-layer WebGIS-NLI framework,
- self-hosted prototype using Ollama,
- schema-based validation of model output,
- deterministic frontend execution,
- system-level explainability metadata,

Natural-language interaction can be connected to WebGIS execution in a bounded and inspectable way.



Thank you

Software Access [GitHub](#)

Contact

Rahaman, M. A.

HFT Stuttgart, Germany.

[in](#) [LinkedIn](#)

Sikder, S. K.,

Leibniz-IOER, Germany

[id](#) [ORCID](#)

[in](#) [LinkedIn](#)



Study Objective

To develop a **locally deployable, chat-driven WebGIS** in which the **LLM proposes bounded actions**, while validation and deterministic execution remain under system control.

Need to combine five properties

WebGIS-oriented NLI

Local open-source LLM deployment

Explicit action validation

System-level explainability

Predefined action set

Limitations and outlook

Just a small effort

- limited action set
- tested for point features
- one local model/configuration
- CPU-only inference
- no formal user study

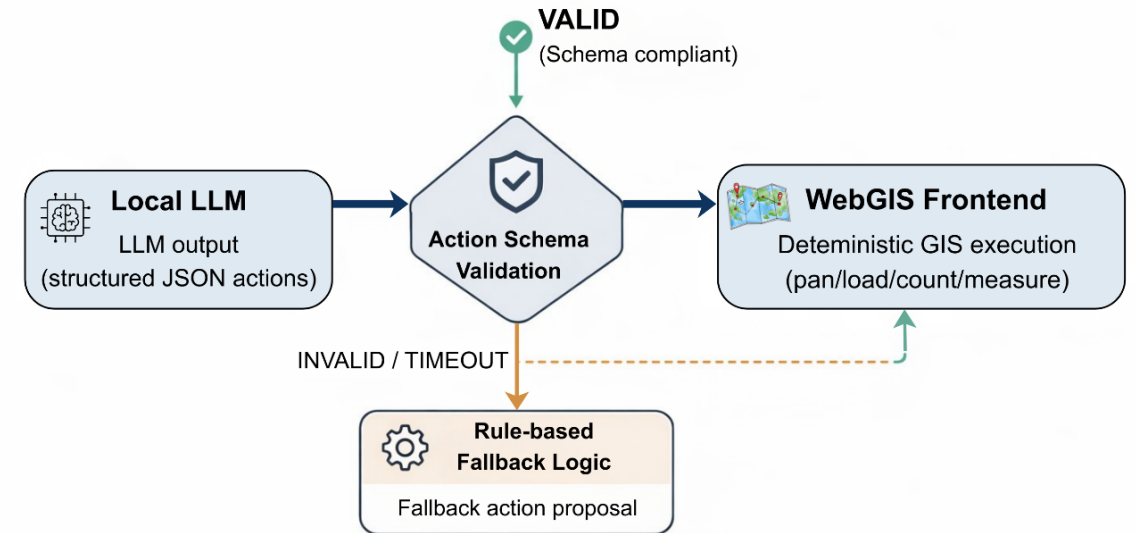
Some scopes

- Features: line, polygon, multi-layer
- Geoprocessing: overlays, routing
- Tests: task success and latency
- User study: usability and trust
- Topics to explore: MCP, A2A

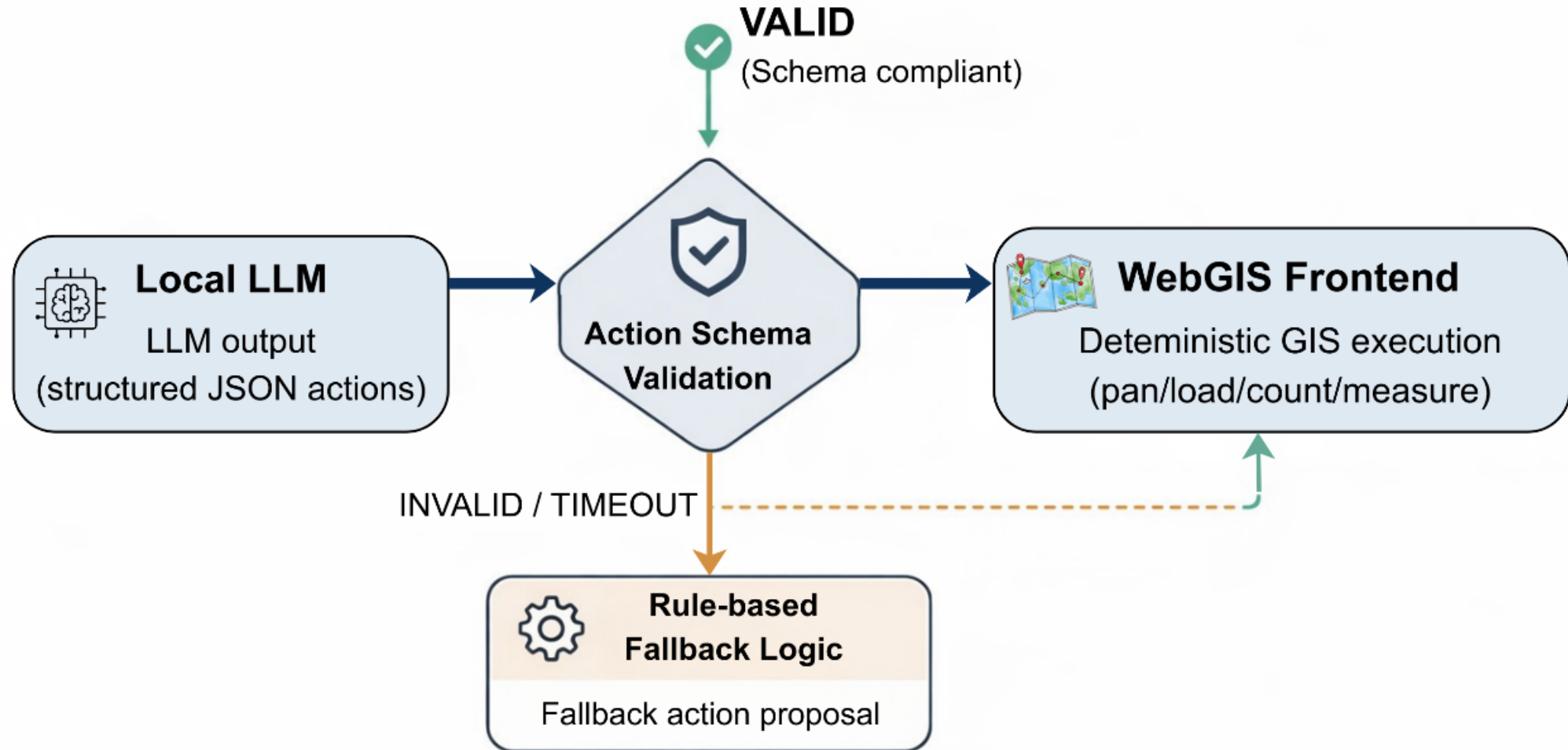
Validation & Fallback

Execution boundary **User request**

1. Local LLM proposes JSON action(s)
2. Action schema checks type and parameters
 - valid: execute in the frontend
 - invalid/timeout: apply rule-based fallback
3. Return map result and explainability metadata



Controlled validation and fallback



Supported actions

The prototype uses a bounded action set.

Action	Purpose
<code>setView</code>	navigate to a place or coordinate
<code>loadWFS</code>	load data from a Web Feature Service
<code>filterWithin</code>	highlight features within a radius
<code>findByAttribute</code>	find features by attribute value
<code>describeLayer</code>	describe the active layer
<code>countLayer</code>	count active-layer features
<code>measureDistance</code>	calculate and show distance
<code>openAttributeTable</code>	open the attribute table

References

- Lin, Q. et al. (2024). *ShapefileGPT: A Multi-Agent Large Language Model Framework for Automated Shapefile Processing*.
<https://doi.org/10.1080/17538947.2025.2577884>
- Mansourian, A. & Oucheikh, R. (2024). *ChatGeoAI: Enabling Geospatial Analysis for Public through Natural Language, with Large Language Models*.
<https://doi.org/10.3390/ijgi13100348>
- Naveen, P. et al. (2024). *GeoNLU: Bridging the gap between natural language and spatial data infrastructures*.
<https://doi.org/10.1016/j.aej.2023.12.027>
- Unrau, R. & Kray, C. (2021). *Mining Map Interaction Semantics in Web-based Geographic Information Systems for Usability Analysis*.
<https://doi.org/10.5194/agile-giss-2-16-2021>
- Wang, S. et al. (2024). *GPT, LLMs and generative AI models in geospatial science: a systematic review*.
<https://doi.org/10.1080/17538947.2024.2353122>
- Zhang, Y. et al. (2024). *MapGPT: An autonomous framework for mapping by integrating a large language model and cartographic tools*.
<https://doi.org/10.1080/15230406.2024.2404868>
